QueryQuest: Database Management Systems

Assignment 1 Solutions

PART I: SHORT ANSWER QUESTIONS

Q1. DBMS vs. Traditional File System

Traditional File System: Stores data as raw OS managed files with no built-in query support, concurrent access control, or integrity enforcement. Each application manages its own file format and access logic, leading to data redundancy and inconsistency.

DBMS: Provides a structured environment with SQL querying, transaction management (ACID properties), access control, and relationship management between data entities.

Scenario: Hospital patient records across departments like doctors, nurses, billing, reception:

- Multiple users update the same patient record simultaneously , a DBMS handles concurrency via locking and transactions, preventing data corruption.
- Queries like "find all diabetic patients admitted in the last 30 days" are trivially expressed in SQL but nearly impossible with flat files.
- A DBMS enforces integrity (e.g., an appointment cannot be created for a non-existent patient via foreign key constraints).
- Role-based access control ensures nurses cannot access billing data, which is impossible to enforce reliably in a file system.

Q2. Weak Entity vs. Strong Entity in ER Diagrams

Aspect	Strong Entity	Weak Entity
Existence	Exists independently	Depends on an owner (identifying) entity
Primary Key	Has its own complete primary key	Has only a partial key (discriminator)
ER Notation	Single rectangle	Double rectangle
Relationship	Regular diamond	Double diamond (identifying relationship)

Primary Key Formation: Primary key of weak entity = Partial key of weak entity + Primary key of identifying entity.
Example: Dependent (weak) identified by Employee (strong). Primary key of Dependent = (emp_id, dependent_name). The relationship connecting a weak entity to its owner is called an identifying relationship.

Q3. Modeling Student Phone Numbers

Answer: Multivalued Attribute.

Justification:

- A simple attribute holds exactly one atomic value — cannot represent a student with multiple numbers.
- A composite attribute is decomposable into sub-parts (e.g., address -> street, city, PIN) — phone numbers have no meaningful sub-components here.
- A multivalued attribute can hold multiple values for one entity instance, which exactly matches the scenario: a student may have zero, one, or several phone numbers.

Q4. Candidate Key vs. Super Key

Super Key: Any set of one or more attributes that uniquely identifies a tuple. May contain redundant attributes.
Candidate Key: A minimal super key; no proper subset of it is also a super key. Every candidate key is a super key, but not vice versa.

Relation: Student (roll no., email, name, branch)
Assumptions: roll no. and email are unique per student; name and branch are not necessarily unique.

Type	Keys	Reason
Candidate Keys	{roll no.}, {email}	Each alone uniquely identifies a student; neither contains redundant attributes.
Super Keys (not candidate)	{roll no., email}	Removing either attribute still gives a unique identifier — redundant.
Super Keys (not candidate)	{roll no., name}	roll_no alone suffices; name is redundant.
Super Keys (not candidate)	{email, branch}	email alone suffices; branch is redundant.

Q5. Constraint Violated When Inserting Non-Existent Student ID

Constraint Violated: Referential Integrity Constraint (Foreign Key Constraint). The student id in Enrollment is a foreign key referencing Student (student id).
What It Protects Against: It prevents orphan records, enrollment entries that reference students who do not exist, keeping the database logically consistent and semantically meaningful.
What Should Happen: The INSERT operation is rejected and aborted. The DBMS raises a foreign key violation error and does not insert the row.

Q6. Mapping a Many-to-Many (M: N) Relationship

Additional Structure: A junction table (also called associative table or bridge table) must be created.
Primary Key: A composite primary key consisting of the foreign keys of both participating entity tables.
Primary key: (student id, course id).
Why a single column won't work: The relational model requires atomicity (1NF) — each column holds exactly one value. Storing multiple course IDs in a single column of Student, or multiple student IDs in a column of Course, would violate 1NF. Because the M:N relationship means one student can have many courses AND one course can have many students, neither side can represent all associations as a flat column — a separate junction table is the only correct relational representation.

PART II: LONG ANSWER QUESTION

Scenario: Online food delivery platform storing Customers, Restaurants, Menu Items, and Orders.

Part A – Entity and Relationship Identification

Entity	Primary Key	Other Attributes
Customer	customer_id	name, email, phone, address
Restaurant	restaurant_id	name, location, cuisine_type
Menu Item	item_id	name, description, price
Order	order_id	order_date, total_amount, status

Relationship	Cardinality	Justification
Customer -> places -> Order	1 : N	One customer can place many orders, but each order belongs to exactly one customer.
Order -> placed_at -> Restaurant	N : 1	Each order is associated with exactly one restaurant, but a restaurant can receive many orders.
Order <-> contains <-> MenuItem	M : N	An order can contain multiple menu items, and the same menu item can appear in multiple orders.

Restaurant -> offers -> MenuItem	1 : N	A restaurant offers multiple menu items, and each menu item belongs to one restaurant.

Part B – Attribute Classification

Multivalued Attribute: Customer's Phone Number
A customer may have more than one phone number (e.g., mobile and home). Since a single customer instance can be associated with multiple phone number values, this is a multivalued attribute. In an ER diagram it is represented with a double ellipse. A single-valued attribute cannot capture this, and a composite attribute refers to decomposition into sub-parts, not multiple instances of the same attribute.

Composite Attribute: Customer's Address
An address is naturally decomposable into meaningful, independently useful sub-components: house/flat number, street, city, state, and PIN code. Each sub-part may need to be accessed individually (e.g., filtering customers by city, or sorting by PIN code for delivery zones). This qualifies it as a composite attribute, shown in the ER diagram with sub-ellipses branching off the main Address ellipse.

Part C – Relational Schema for Junction Table

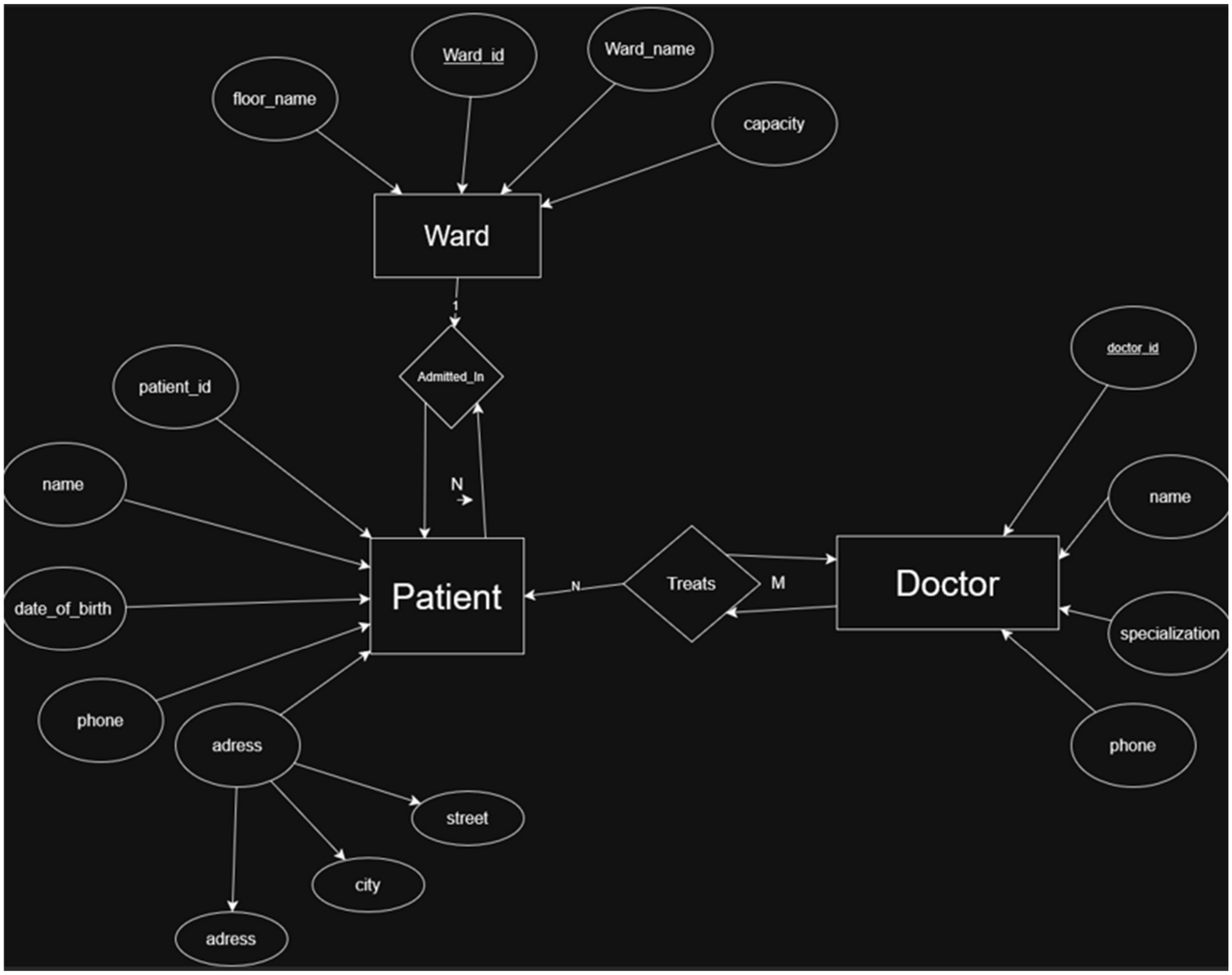
The M:N relationship between Order and Menu Item requires a junction table:

Element	Detail	Description
Primary Key	(order_id, item_id)	Composite; uniquely identifies each line item within an order.
Foreign Key 1	order_id	References Order(order_id)
Foreign Key 2	item_id	References MenuItem(item_id)
Additional Attribute	quantity (INT, NOT NULL)	Stores how many units of a menu item are in the order; this attribute belongs to the relationship itself, not to either entity alone.

PART III: ER Diagram Design

Question 1: Hospital ER Diagram

Below is a description of the ER diagram (drawn in draw.io):



Question 2: Design Justification

Doctor–Patient as M:N: The relationship between Doctor and Patient is M:N because a single doctor can treat multiple patients over the course of their stay, and a single patient may be attended to by multiple doctors (e.g., a surgeon, a cardiologist, and a general physician all treating the same patient). This M:N relationship gives rise to an associative entity called Treatment (or Doctor_Patient). Its composite primary key would consist of (doctor_id, patient_id), and it may additionally carry attributes such as treatment_date or diagnosis_notes.

Participation constraints: In the *Admitted_In* relationship, Patient has total participation because every patient in the hospital system must be assigned to a ward — a patient cannot logically exist in the database without being housed somewhere. Ward has partial participation because a ward can exist in the system (perhaps newly built or temporarily empty) without having any patient currently assigned to it.

Attribute design decision: The Patient's address was modeled as a composite attribute (broken into street, city, and pin code) rather than a single string. This allows the hospital to filter or group patients by city for administrative or emergency reporting purposes, which would be difficult if the address were stored as one opaque value.